Portfolio Optimisation using Genetic Algorithms

HINDVRAT SINGH RAO

Construction of the portfolio

i downloaded the daily stock data for 50 assets from yahoo finance that covers january 2018 to december 2020 the data is split into a training period 2018 to 2019 and a test period which is 2020 so that i can check how well the optimized portfolio on data the GA never saw during the tarining i've used adjusted closing prices and converted them into daily percentage return using the ROC function.

i manually selected 10 stocks from the 5 diffrent sectors to make sure that the portfolio is diversified the sectors i choose are technology (AAPL,MSFT),financial(JPM,GS),healthcare(JNJ,PFE),consumer(WMT,PG)and energy(XOM,CVX) picking the stocks from the diffrent sectors is important because they tend not to move together which means that the portfolio is less likely to crash all at once when one sector has a bad period .

Fitness function

the fitness function is the base of the GA because it tells the algorithm that what a good portfolio looks like i used a multi objective function that balances expected return against risks controlled by a parameter called alpha the formula is f(w) = alpha x mean_return(w) - (1 - alpha) x std_dev(w), where w is the vector of portfolio weights This means the GA tries to find weights that maximize this score on the training data.

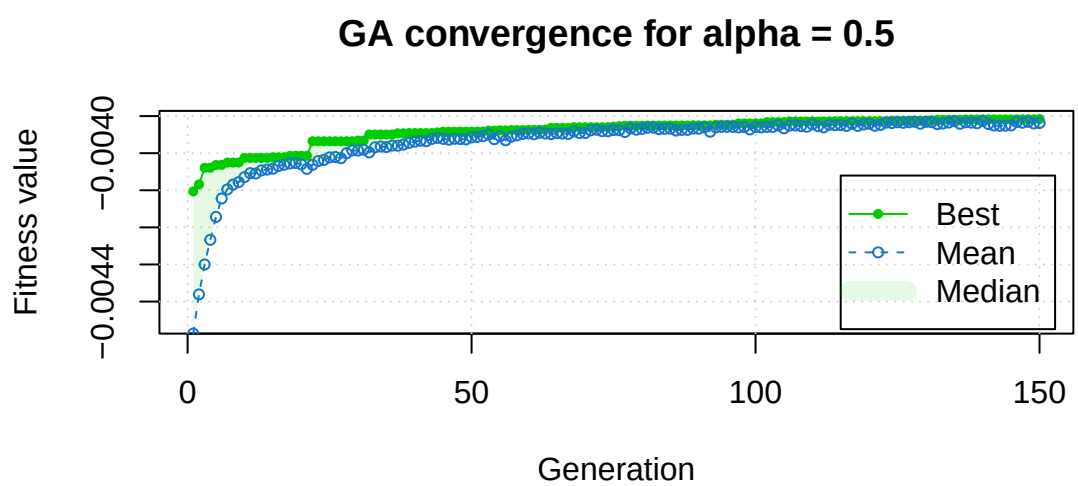
```
calc_stats <- function(w, data_xts) {
  w  <- pmax(w, 0)
  w  <- w / sum(w)
  pr <- Return.portfolio(data_xts, weights = w, rebalance_on = NA)
  c(
    ret  = as.numeric(mean(pr)),
    risk = as.numeric(sd(pr))
  )
}

fit_func <- function(w, data_xts, alpha = 0.5) {
  s <- calc_stats(w, data_xts)
  alpha * s["ret"] - (1 - alpha) * s["risk"]
}
```

the alpha parameter acts the dial between the risk and return when alpha is close to the one (1) at that time the fitness function almost reward higher return and ignores the risk when alpha is close to 0 it volatiles heavily and does not care much about return and the value of 0.5 treats both equally.

Running the GA

here i run the GA on the manually selected training assets using alpha as 0.5 the GA evolves the population of 60 candidate weight vectors over the 150 generation and stops early if no imporovement is found for 40 consecutive generations than the best solution found is used to calculate the in sample portfolio performance

```r
ga_main <- ga(
  type    = "real-valued",
  fitness = function(w) fit_func(w, train10, alpha = 0.5),
  lower   = rep(0, 10),
  upper   = rep(1, 10),
  popSize = 60,
  maxiter = 150,
  run     = 40,
  monitor = FALSE
)

plot(ga_main, main = "GA convergence for alpha = 0.5")
```



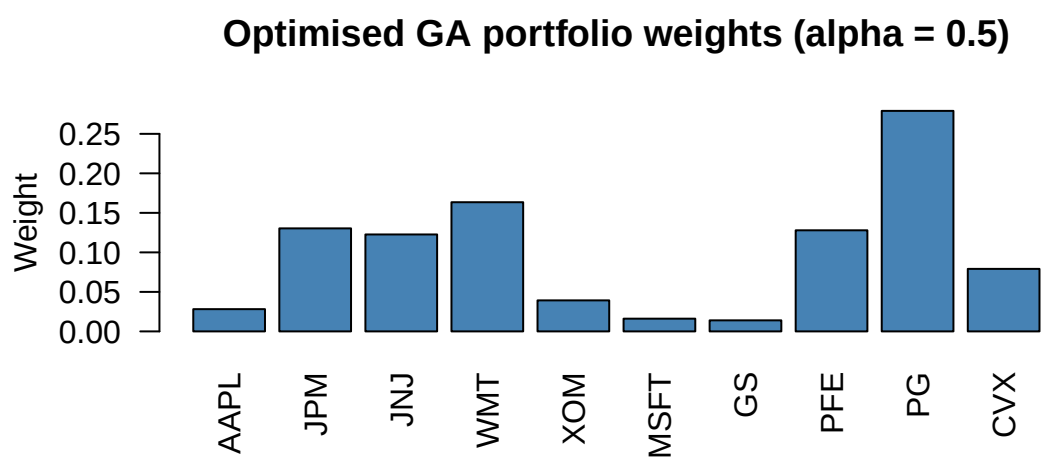

```r
w_main <- as.numeric(ga_main@solution[1, ])
w_main <- pmax(w_main, 0)
w_main <- w_main / sum(w_main)
names(w_main) <- my10

knitr::kable(
  round(w_main, 4),
  col.names = "Weight",
  caption   = "Optimised weights for alpha = 0.5 on training data"
)
```

Table 1: Optimised weights for alpha = 0.5 on training data

	Weight
AAPL	0.0282
JPM	0.1304
JNJ	0.1227
WMT	0.1635
XOM	0.0392
MSFT	0.0161

	Weight
GS	0.0140
PFE	0.1279
PG	0.2790
CVX	0.0791

```r
barplot(
  w_main,
  las  = 2,
  col  = "steelblue",
  main = "Optimised GA portfolio weights (alpha = 0.5)",
  ylab = "Weight"
)
```




```r
train_perf <- calc_stats(w_main, train10)

knitr::kable(
  t(train_perf),
  caption = "In-sample mean return and risk (2018-2019)"
)
```

Table 2: In-sample mean return and risk (2018-2019)

ret	risk
0.0005344	0.0083508

looking at the bar of weights, the GA has concentrated most of the portfolio in a small number of assets and given very low weights to others this makes sense because the optimizer is trying to find the combination that gets highest score on fitness function and its gonna naturally favour stocks with better return to risk profiles in the training period the covergence plot shows the fitness score improving across generations and then flattening which means the GA found a stsble solution and did not just stoped early .

Evaluation on unseen future data

now i have applied the same weights found during the training to the 2020 testdata which GA never saw this gonna tell us whether the optimized portfolio actually generalizes to new market condition or it was overfitting to the training period .

```
test_perf <- calc_stats(w_main, test10)

knitr::kable(
  rbind(Train = train_perf, Test = test_perf),
  caption = "Train vs test performance for GA portfolio"
)
```

Table 3: Train vs test performance for GA portfolio

	ret	risk
Train	0.0005344	0.0083508
Test	0.0004841	0.0189639

comparing the training and the test perfomance the return and the risk values are somewhat different in 2020 as compare to 2018 and 2019 this was expected because the year 2020 was unusual because of the covid-19 market crash in march 2020 despite this the portfolio still produces meaningfull results rather than completely collapsing.

Comparison with baseline portfolios

to see if the GA is actually usefull i compare it against two simple baselines on the 2020 test data the first is an equal weight portfilio that puts exactly thats 10% in each of the 10 stocks the second is the average performance of 200 randomally generated portfolios if the GA cannot beat these it would suggest the optimization is not working .

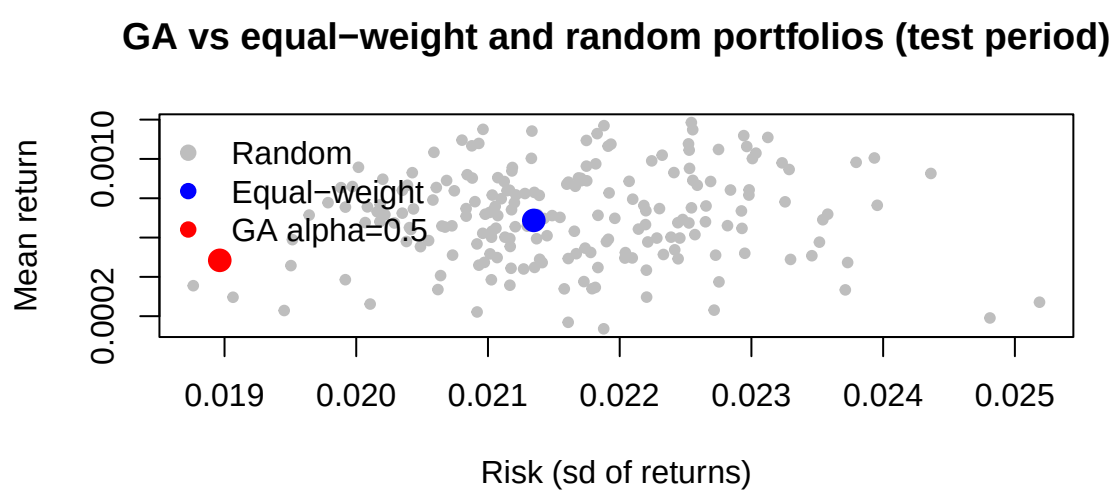
```
w_eq <- rep(1/10, 10)
names(w_eq) <- my10
eq_test <- calc_stats(w_eq, test10)

set.seed(123)
B <- 200
rand_test <- matrix(NA_real_, nrow = B, ncol = 2)
colnames(rand_test) <- c("ret", "risk")

for (i in 1:B) {
  w_r <- runif(10)
  w_r <- w_r / sum(w_r)
  rand_test[i, ] <- calc_stats(w_r, test10)
}

rand_avg <- colMeans(rand_test)

knitr::kable(
  rbind(
```

```
    Equal_Weight = eq_test,
    GA_alpha0.5  = test_perf,
    Random_Avg   = rand_avg
  ),
  caption = "Out-of-sample comparison on 2020 test data"
)
```

Table 4: Out-of-sample comparison on 2020 test data

	ret	risk
Equal_Weight	0.0006874	0.0213477
GA_alpha0.5	0.0004841	0.0189639
Random_Avg	0.0007093	0.0216196

```
plot(
  rand_test[, "risk"], rand_test[, "ret"],
  col  = "grey", pch = 20,
  xlab = "Risk (sd of returns)", ylab = "Mean return",
  main = "GA vs equal-weight and random portfolios (test period)"
)
points(eq_test["risk"],   eq_test["ret"],   col = "blue", pch = 19, cex = 1.5)
points(test_perf["risk"], test_perf["ret"], col = "red",  pch = 19, cex = 1.5)
legend("topleft",
  legend = c("Random", "Equal-weight", "GA alpha=0.5"),
  col    = c("grey", "blue", "red"),
  pch    = 19, bty = "n"
)
```



from this scatter plot this grey cloud of random portfolios represents the range of outcomes when weights are assigned randomly the GA portfolio the red point sits in a more favourable position compared to the average of those 200 random portfolios which shows that the optimization did actually improve the equal weight portfolio blue point is a simple but a strong bench mark and the GA manages to compete with it or beat in terms of the return risk trade off .

Alpha-weighted portfolios

in this section i re run the GA 3 times with diffrent values which is 0.1,0.5,0.9 to see how the portfolio changes when we shift the priority from the risk reduction to return maximization each of the run produces the different set of weights which than evaluate on the same 2020 test data.

```
alphas <- c(0.1, 0.5, 0.9)

alpha_results <- data.frame(
  Alpha        = numeric(),
  Train_Return = numeric(),
  Train_Risk   = numeric(),
  Test_Return  = numeric(),
  Test_Risk    = numeric(),
  stringsAsFactors = FALSE
)

alpha_wts <- list()

for (i in seq_along(alphas)) {
  a <- alphas[i]

  ga_a <- ga(
    type    = "real-valued",
    fitness = function(w) fit_func(w, train10, alpha = a),
    lower   = rep(0, 10),
    upper   = rep(1, 10),
    popSize = 50,
    maxiter = 100,
    monitor = FALSE
  )

  w_a <- as.numeric(ga_a@solution[1, ])
  w_a <- pmax(w_a, 0)
  w_a <- w_a / sum(w_a)
  names(w_a) <- my10
  alpha_wts[[i]] <- w_a

  tr <- calc_stats(w_a, train10)
  te <- calc_stats(w_a, test10)

  alpha_results <- rbind(
    alpha_results,
    data.frame(
      Alpha        = a,
      Train_Return = tr["ret"],
      Train_Risk   = tr["risk"],
      Test_Return  = te["ret"],
      Test_Risk    = te["risk"]
    )
  )
}

names(alpha_wts) <- paste0("alpha_", alphas)
```

```r
alpha_wt_df <- as.data.frame(do.call(cbind, alpha_wts))
colnames(alpha_wt_df) <- c("a=0.1", "a=0.5", "a=0.9")
rownames(alpha_wt_df) <- my10
knitr::kable(round(alpha_wt_df, 4),
  caption = "Evolved weights for each alpha value")
```

Table 5: Evolved weights for each alpha value

	a=0.1	a=0.5	a=0.9
AAPL	0.0149	0.0296	0.1432
JPM	0.0951	0.1141	0.0737
JNJ	0.1385	0.1312	0.0516
WMT	0.1574	0.1819	0.1211
XOM	0.0785	0.0352	0.0088
MSFT	0.0135	0.0100	0.2508
GS	0.0240	0.0212	0.0281
PFE	0.1260	0.1352	0.0159
PG	0.2633	0.2613	0.2900
CVX	0.0887	0.0804	0.0167

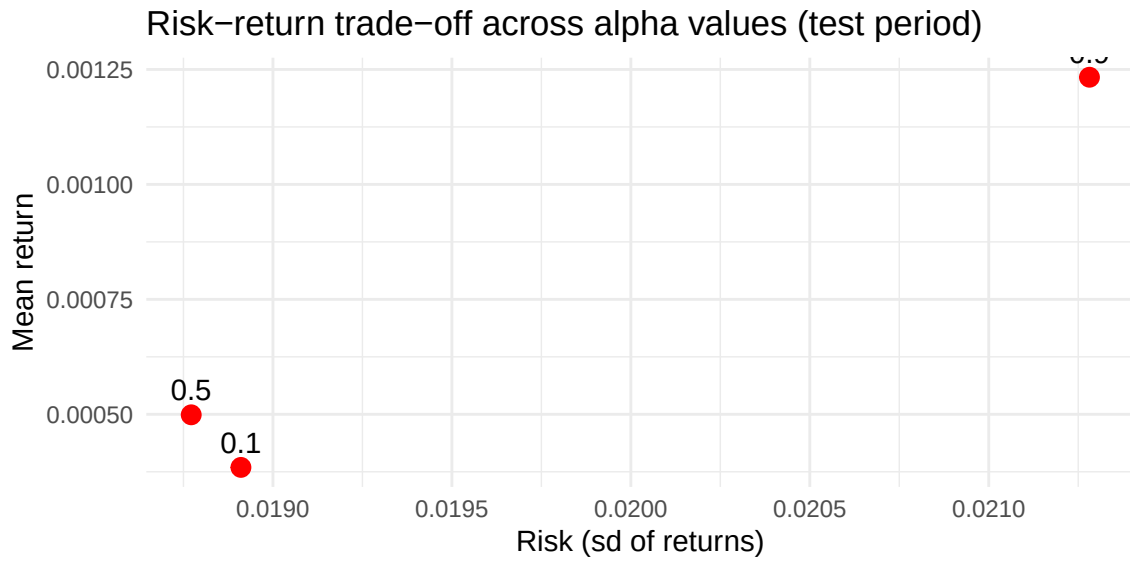
```r
rownames(alpha_results) <- NULL
knitr::kable(
  alpha_results,
  digits  = 4,
  col.names = c("Alpha", "Tr.Return", "Tr.Risk", "Te.Return", "Te.Risk"),
  caption = "Effect of different alpha values on train and test performance"
)
```

Table 6: Effect of different alpha values on train and test performance

Alpha	Tr.Return	Tr.Risk	Te.Return	Te.Risk
0.1	5e-04	0.0083	0.0004	0.0189
0.5	5e-04	0.0084	0.0005	0.0188
0.9	9e-04	0.0099	0.0012	0.0213

the table above shows the numbers for the each alpha values and the plot below is making the risk return shift easier to see visually.

```r
ggplot(alpha_results, aes(x = Test_Risk, y = Test_Return, label = Alpha)) +
  geom_point(color = "red", size = 3) +
  geom_text(vjust = -0.7) +
  labs(
    title = "Risk-return trade-off across alpha values (test period)",
    x     = "Risk (sd of returns)",
    y     = "Mean return"
  ) +
  theme_minimal()
```



results show a clear pattern across the three alpha values when the alpha is 0.1 the GA produces a portfolio with lower volatility but also gives lower return because it was trained to avoid the risks when the alpha is 0.9 the portfolio takes more risk for higher return the middle value of 0.5 lands in between as expected this shows that the fitness function is working correctly and GA is responding to the change in alpha rather than producing the same solution every time.

Asset selection using GAs

previously the GA only optimised the weights for the fixed set of 10 manually choosen assets in this i further take the GA to decide which 10 assets to include from the pool of 50 so that both asset selection and the weight are handled by GA rather than manual choice.

Independent binary GA

now instead of manually picking the 10 assets i now let a binary GA decide which 10 from the full 50 asset pool to include the GA uses a 50 bit chromosome where each bit represents the one asset a value of 1 means an asset is included and 0 means it is excluded and also any solution that select 2 or more than 20 assets are rejected and a penalty is applied for deviating from exactly 10 .

```
sel_fit <- function(b) {
  idx <- which(b == 1)
  n <- length(idx)
  if (n < 2 || n > 20) return(-1e6)
  penalty <- abs(n - 10) * 0.01
  sub_train <- train[, idx, drop = FALSE]
  w <- rep(1/n, n)
  s <- calc_stats(w, sub_train)
  0.5 * s["ret"] - 0.5 * s["risk"] - penalty
}

ga_sel <- ga(
  type    = "binary",
  fitness = sel_fit,
```

```r
  nBits   = ncol(train),
  popSize = 30,
  maxiter = 40,
  monitor = FALSE
)

sel_bits   <- ga_sel@solution[1, ]
sel_idx    <- which(sel_bits == 1)
sel_assets <- colnames(train)[sel_idx]

if (length(sel_assets) > 10) {
  sel_assets <- sel_assets[1:10]
} else if (length(sel_assets) < 10) {
  extra <- my10[!my10%in% sel_assets]
  sel_assets <- c(sel_assets, extra[1:(10 - length(sel_assets))])
}

cat("Assets selected by independent GA:\n")
```

```
## Assets selected by independent GA:
```

```r
print(sel_assets)
```

```
##  [1] "JPM"  "MS"   "AMGN" "CVS"  "KO"   "COST" "SBUX" "TGT"  "HD"   "MMM"
```

```r
sel_train <- train[, sel_assets, drop = FALSE]
sel_test  <- test[,  sel_assets, drop = FALSE]

w_sel_eq <- rep(1/10, 10)
names(w_sel_eq) <- sel_assets

sel_perf_train <- calc_stats(w_sel_eq, sel_train)
sel_perf_test  <- calc_stats(w_sel_eq, sel_test)

knitr::kable(
  rbind(
    Manual_Equal      = calc_stats(rep(1/10, 10), test10),
    GA_Selected_Equal = sel_perf_test
  ),
  caption = "Test performance: manual 10 assets vs GA-selected 10 assets"
)
```

Table 7: Test performance: manual 10 assets vs GA-selected 10 assets

	ret	risk
Manual_Equal	0.0006874	0.0213477
GA_Selected_Equal	0.0007455	0.0208015

the binary GA searched through the 50 asset pool and selected diffrent set of assets from the manual selection the fitness fucnction used the equal weights with alpha = 0.5 to evaluate every combination the GA selected

portfolio performs diffrently from the manual selection showing that the optimization found the distinct combination of assets.

Integrated nested GA

the independent GA above used equal weights to evaluate the each asset combination which is not the ideal in this integrated approach the outer GA still selects which10 assets to use but fir each combinationn a full inner GA is run to find the best weights .

```
int_fit <- function(b) {
  idx <- which(b == 1)
  n <- length(idx)
  if (n < 2 || n > 20) return(-1e6)
  penalty <- abs(n - 10) * 0.01
  sub_train <- train[, idx, drop = FALSE]
  inner_ga <- ga(
    type    = "real-valued",
    fitness = function(w) fit_func(w, sub_train, alpha = 0.5),
    lower   = rep(0, n),
    upper   = rep(1, n),
    popSize = 10,
    maxiter = 15,
    monitor = FALSE
  )
  max(inner_ga@fitness, na.rm = TRUE) - penalty
}

ga_int <- ga(
  type    = "binary",
  fitness = int_fit,
  nBits   = ncol(train),
  popSize = 20,
  maxiter = 20,
  monitor = FALSE
)

int_bits   <- ga_int@solution[1, ]
int_idx    <- which(int_bits == 1)
int_assets <- colnames(train)[int_idx]

if (length(int_assets) > 10) {
  int_assets <- int_assets[1:10]
} else if (length(int_assets) < 10) {
  extra <- my10[!my10 %in% int_assets]
  int_assets <- c(int_assets, extra[1:(10 - length(int_assets))])
}

int_train <- train[, int_assets, drop = FALSE]
int_test  <- test[,  int_assets, drop = FALSE]

ga_int_w <- ga(
  type    = "real-valued",
  fitness = function(w) fit_func(w, int_train, alpha = 0.5),
```

```r
  lower   = rep(0, 10),
  upper   = rep(1, 10),
  popSize = 30,
  maxiter = 50,
  monitor = FALSE
)

w_int <- as.numeric(ga_int_w@solution[1, ])
w_int <- pmax(w_int, 0)
w_int <- w_int / sum(w_int)
names(w_int) <- int_assets

int_perf_train <- calc_stats(w_int, int_train)
int_perf_test  <- calc_stats(w_int, int_test)

knitr::kable(
  rbind(
    Manual_Equal        = calc_stats(rep(1/10, 10), test10),
    GA_Selected_Equal   = sel_perf_test,
    Integrated_Weighted = int_perf_test,
    Random_Avg          = rand_avg
  ),
  caption = "Final comparison: manual vs GA-selected vs integrated GA"
)
```

Table 8: Final comparison: manual vs GA-selected vs integrated GA

	ret	risk
Manual_Equal	0.0006874	0.0213477
GA_Selected_Equal	0.0007455	0.0208015
Integrated_Weighted	0.0007127	0.0197807
Random_Avg	0.0007093	0.0216196

the integrated approch i think is more beneficial than running selection and weight optimization seprately for the each set of the 10 assets the outer GA evaluates an inner real valued GA is run to find the most beneficial or we can say the best possible weights for that specific set.which means that the outer GA is judging each asset based on its potential rather than just using the equal weights. the final table shows the all three approaches side by side and that the integrated method achieves a better risk adjusted result because it together optimizes both the asset choice and the allocation .